IF25-22014 PENGEMBANGAN APLIKASI WEB

Pertemuan 12
Basis Data PostgreSQL

Menyimpan data proyek secara permanen dan memakainya dari FastAPI

Tim Pengajar
Tahun 2026. Rujukan: Dokumentasi PostgreSQL (`postgresql.org/docs`), SQLAlchemy 2.0 (`docs.sqlalchemy.org`), Alembic, dan FastAPI *SQL Databases*.

ITERA

DIKTISAINTEK BERDAMPAK

Program Studi Teknik Informatika
Institut Teknologi Sumatera

PETA PERTEMUAN

Perjalanan kalian hari ini

ITERA

Fase	Menit	Yang kalian lakukan	Pertanyaan wajar	Bila tersendat
Pembuka	0–10	Menjawab tiga soal minggu lalu dengan jari	Masih ingat yang kemarin?	Catat nomor yang salah, cek di modul minggu lalu
Coba dulu	10–30	Mengerjakan tiga misi berpasangan	Boleh salah?	Tulis dugaan; salah itu bahan belajar
Materi	30–85	Mengikuti 6 bagian: tebak, ikuti bersama, engsel	Sedang di bagian mana?	Lihat penanda di atas judul atau klik Outline
Hands-on	85–120	Mengerjakan tingkat 1 sampai 3 dari `starter/`	Sudah benar belum?	Periksa berpasangan; buka modul bagian yang sama
Papan	120–140	Menunjukkan atau menanggapi hasil teman	Kenapa kode itu jalan?	Ajukan satu pertanyaan mengapa
Exit ticket	140–150	Menjawab dua soal tanpa catatan	Apa yang belum paham?	Tulis di kolom refleksi

Penanda fase dan bagian ada di atas setiap judul slide. Kotak **Ikuti bersama** berarti ketik di laptop kalian sekarang.

Soal dari Pertemuan 11

ITERA

Soal 1. Body tidak lolos model Pydantic dibalas?

- **A** 200
- **B** 400
- **C** 404
- **D** 422

Soal 2. Path REST untuk menghapus mata kuliah?

- **A** GET /hapus
- **B** POST /matkul/hapus
- **C** DELETE /matkul/kode
- **D** PUT /matkul

Soal 3. Data API P11 hilang saat server mati karena?

- **A** CORS
- **B** disimpan di list Python
- **C** Pydantic
- **D** port salah

KUNCI

1D: FastAPI membalas 422. 2C: metode menyatakan tindakan. 3B: memori hilang saat proses berhenti.

Server mati, data ikut hilang

ITERA

API mata kuliah P11 menyimpan data di list Python. Setiap kali server dimuat ulang, mata kuliah yang ditambahkan lewat Next.js hilang. Di proyek, data pengguna, KRS, dan nilai harus tetap ada walaupun server mati, dan bisa dipakai beberapa server sekaligus.

PIKIRKAN

Di mana data seharusnya disimpan, dan bagaimana FastAPI membacanya tanpa menulis ulang semua endpoint?

```
DATA: list[dict] = [
    {"kode": "IF25-22014", "sks": 3},
]

@app.post("/matkul", status_code=201)
def tambah(m: MatkulIn):
    DATA.append(m.model_dump())
    return m
# muat ulang server: DATA kembali semula
```

Misi kalian, 20 menit

ITERA

Berdua atau bertiga, pakai apa saja yang kalian tahu. Berkas awal ada di folder hands-on, `starter/`.

1 Masuk

Buka psql atau DBeaver, buat basis data `krs_NIM`, lalu jalankan `starter/schema.sql` dan `seed.sql`.

2 Tanya data

Tulis query yang menampilkan nama mata kuliah ber-SKS 3, urut menurut nama.

3 Gabung

Tampilkan nama mahasiswa beserta nama mata kuliah yang diambilnya. Tabel apa saja yang dibutuhkan?

Boleh

Bertanya ke teman, mencoba di browser, salah.

Belum dulu

Mencari di internet atau bertanya ke AI.

Enam bagian hari ini

ITERA

	Bagian	Tujuan (klik judul untuk melompat)
1	**Basis data relasional dan PostgreSQL**	**Menjelaskan peran basis data relasional di aplikasi web**
2	SQL dasar	Menulis query untuk membuat, membaca, mengubah, dan menghapus data
3	Relasi dan JOIN	Menghubungkan tabel dengan foreign key dan menggabungkannya dengan JOIN
4	SQLAlchemy ORM	Memetakan tabel ke class Python dan menjalankan query dengan session
5	FastAPI dan PostgreSQL	Mengganti list Python dengan basis data di endpoint FastAPI
6	Migrasi dan keamanan	Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query

Penyimpan permanen

PostgreSQL
Sistem basis data relasional sumber terbuka yang berjalan sebagai server terpisah (bawaan port 5432) dan menyimpan data dalam tabel yang saling terhubung.

Ingat peta ekosistem P1: basis data hanya diakses server, tidak pernah langsung oleh browser. FastAPI menjadi satu-satunya pintu; Next.js meminta lewat API.

```
-- tabel: baris dan kolom
CREATE TABLE matkul (
  id SERIAL PRIMARY KEY,
  kode VARCHAR(12) UNIQUE NOT NULL,
  nama TEXT NOT NULL,
  sks INT CHECK (sks BETWEEN 1 AND 6)
);
```

Istilah yang dipakai hari ini

ITERA

Istilah	Artinya
tabel, baris, kolom	satu jenis data; satu data; satu atribut
primary key	kolom unik penanda setiap baris, misalnya `id`
foreign key	kolom yang menunjuk primary key tabel lain
constraint	aturan di tingkat basis data: `NOT NULL`, `UNIQUE`, `CHECK`
transaksi	sekumpulan perubahan yang berhasil semua atau batal semua (`COMMIT`, `ROLLBACK`)

Ingat

Validasi berlapis: formulir React di browser untuk kenyamanan, Pydantic di FastAPI untuk API, constraint di PostgreSQL sebagai pertahanan terakhir.

Ikuti bersama: basis data pertama

ITERA

1. Buat basis data milik kalian.
   `createdb krs_NIM`
2. Masuk dengan psql.
   `psql -d krs_NIM`
3. Jalankan skema dari folder `starter/`.
   `\i schema.sql`
4. Isi data contoh.
   `\i seed.sql`
5. Lihat tabel dan isi satu tabel.
   `\dt`

YANG HARUS TERLIHAT

`\dt` menampilkan tiga tabel: mahasiswa, matkul, krs. `SELECT * FROM matkul;` menampilkan lima mata kuliah.

Kalau berbeda

password authentication failed: pakai pengguna yang dibuat saat pemasangan, misalnya `psql -U postgres -d krs_NIM`. *createdb tidak dikenal*: tambahkan folder bin PostgreSQL ke PATH, atau buat basis data lewat DBeaver.

Enam bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Basis data relasional dan PostgreSQL	Inti: Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	**SQL dasar**	**Menulis query untuk membuat, membaca, mengubah, dan menghapus data**
3	Relasi dan JOIN	Menghubungkan tabel dengan foreign key dan menggabungkannya dengan JOIN
4	SQLAlchemy ORM	Memetakan tabel ke class Python dan menjalankan query dengan session
5	FastAPI dan PostgreSQL	Mengganti list Python dengan basis data di endpoint FastAPI
6	Migrasi dan keamanan	Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query

Empat perintah utama

```
INSERT INTO matkul (kode, nama, sks)
VALUES ('IF25-22014', 'Pengembangan Aplikasi Web', 3);

SELECT kode, nama FROM matkul
WHERE sks = 3
ORDER BY nama
LIMIT 10;

UPDATE matkul SET sks = 4 WHERE kode = 'IF25-22014';

DELETE FROM matkul WHERE kode = 'IF25-21099';
```

`UPDATE` dan `DELETE` tanpa `WHERE` mengenai **semua** baris. Biasakan menulis `WHERE` lebih dulu.

Enam bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Basis data relasional dan PostgreSQL	**Inti:** Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	SQL dasar	**Inti:** INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.
3	**Relasi dan JOIN**	**Menghubungkan tabel dengan foreign key dan menggabungkannya dengan JOIN**
4	SQLAlchemy ORM	Memetakan tabel ke class Python dan menjalankan query dengan session
5	FastAPI dan PostgreSQL	Mengganti list Python dengan basis data di endpoint FastAPI
6	Migrasi dan keamanan	Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query

Tabel penghubung KRS

```
CREATE TABLE mahasiswa (
  id SERIAL PRIMARY KEY, nim CHAR(9) UNIQUE, nama TEXT
);
CREATE TABLE krs (
  mahasiswa_id INT REFERENCES mahasiswa(id) ON DELETE CASCADE,
  matkul_id INT REFERENCES matkul(id),
  semester INT,
  PRIMARY KEY (mahasiswa_id, matkul_id, semester)
);

SELECT m.nama, k.nama, k.sks
FROM krs
JOIN mahasiswa m ON m.id = krs.mahasiswa_id
JOIN matkul k ON k.id = krs.matkul_id
WHERE krs.semester = 3;
```

`JOIN ... ON`: gabungkan baris yang kolom penunjuknya cocok. Jawaban misi 3.

Total SKS per mahasiswa

ITERA

Query mana yang menghasilkan total SKS setiap mahasiswa?

A `SELECT SUM(sks) FROM matkul`

B `... GROUP BY m.nama` dengan `SUM(k.sks)`

C `... ORDER BY m.nama`

D `... WHERE SUM(k.sks)`

JAWABAN: B

JOIN tiga tabel lalu `GROUP BY` mahasiswa dengan `SUM(k.sks)`. Syarat pada hasil penjumlahan memakai `HAVING`, bukan `WHERE`.

Enam bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Basis data relasional dan PostgreSQL	**Inti:** Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	SQL dasar	**Inti:** INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.
3	Relasi dan JOIN	**Inti:** Foreign key menghubungkan tabel; banyak-ke-banyak lewat tabel penghubung; JOIN.
4	**SQLAlchemy ORM**	**Memetakan tabel ke class Python dan menjalankan query dengan session**
5	FastAPI dan PostgreSQL	Mengganti list Python dengan basis data di endpoint FastAPI
6	Migrasi dan keamanan	Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query

Tabel sebagai class

ITERA

ORM (Object Relational Mapping)
Teknik memetakan tabel basis data ke class dan baris ke object, sehingga query ditulis sebagai kode Python dan diterjemahkan menjadi SQL oleh pustaka seperti SQLAlchemy.

OOP P10 bertemu basis data: `Matkul` adalah class, setiap baris adalah object. Query dengan parameter juga mencegah SQL injection.

```
from sqlalchemy import create_engine, String
from  sqlalchemy.orm  import  DeclarativeBase,
Mapped, mapped_column

engine = create_engine("postgresql+psycopg://web:rahas

class Base(DeclarativeBase): pass

class Matkul(Base):
    __tablename__ = "matkul"
    id:                 Mapped[int]         =
mapped_column(primary_key=True)
    kode:                Mapped[str]        =
mapped_column(String(12), unique=True)
    nama: Mapped[str]
    sks: Mapped[int]
```

Session: membaca dan menulis

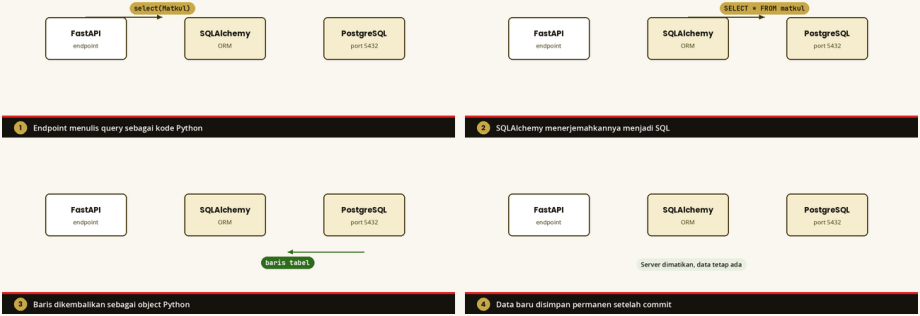
```
from sqlalchemy import select
from sqlalchemy.orm import Session

with Session(engine) as db:
    q = select(Matkul).where(Matkul.sks == 3)
    q = q.order_by(Matkul.nama)
    semua = db.scalars(q).all()
    satu = db.get(Matkul, 1)

    baru = Matkul(kode="IF25-21030",
                  nama="AI", sks=3)
    db.add(baru)
    db.commit() # simpan permanen
    db.refresh(baru) # isi id baru
```

`add` lalu `commit`: INSERT baru tersimpan setelah commit. Tanpa commit, perubahan dibatalkan saat session ditutup.

Dari kode ke SQL



SQLAlchemy menerjemahkan kode Python menjadi SQL dan baris tabel menjadi object; inilah jawaban masalah minggu ini.

Enam bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Basis data relasional dan PostgreSQL	**Inti:** Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	SQL dasar	**Inti:** INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.
3	Relasi dan JOIN	**Inti:** Foreign key menghubungkan tabel; banyak-ke-banyak lewat tabel penghubung; JOIN.
4	SQLAlchemy ORM	**Inti:** ORM: tabel jadi class, baris jadi object; session menjalankan query dan transaksi.
5	**FastAPI dan PostgreSQL**	**Mengganti list Python dengan basis data di endpoint FastAPI**
6	Migrasi dan keamanan	Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query

Endpoint dengan session

ITERA

```
DB = Annotated[Session, Depends(get_db)] # get_db membuka
Session

class MatkulOut(MatkulIn):
    model_config = ConfigDict(from_attributes=True)
    id: int

@app.get("/matkul", response_model=list[MatkulOut])
def daftar(db: DB):
    return db.scalars(select(Matkul)).all()

@app.post("/matkul",                response_model=MatkulOut,
status_code=201)
def tambah(m: MatkulIn, db: DB):
    baru = Matkul(**m.model_dump())
    db.add(baru); db.commit(); db.refresh(baru)
    return baru
```

POST: buat object dari model masukan, simpan, refresh agar `id` terisi. Endpoint lain di Next.js tidak perlu berubah sama sekali.

Enam bagian hari ini

ITERA

	Bagian	Tujuan, dan inti bagian yang sudah dibahas
1	Basis data relasional dan PostgreSQL	**Inti:** Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	SQL dasar	**Inti:** INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.
3	Relasi dan JOIN	**Inti:** Foreign key menghubungkan tabel; banyak-ke-banyak lewat tabel penghubung; JOIN.
4	SQLAlchemy ORM	**Inti:** ORM: tabel jadi class, baris jadi object; session menjalankan query dan transaksi.
5	FastAPI dan PostgreSQL	**Inti:** Dependency memberi session per request; Pydantic membaca object ORM menjadi JSON.
6	**Migrasi dan keamanan**	**Mengelola perubahan skema dengan Alembic dan menjaga kredensial serta query**

Alembic

```
pip install alembic
alembic init alembic
# alembic/env.py: target_metadata = Base.metadata

alembic revision --autogenerate -m "tambah kolom semester"
alembic upgrade head # terapkan ke basis data
alembic downgrade -1 # batalkan satu langkah
```

`downgrade` membatalkan migrasi terakhir bila ada yang salah.

Tiga baris yang salah

ITERA

Yang ditulis	Mengapa salah, dan perbaikannya
query dirakit dengan f-string dari NIM masukan pengguna	SQL injection: masukan bisa berisi perintah SQL. Pakai ORM atau parameter.
password basis data ditulis langsung di `create_engine`	Password ikut ke GitHub. Simpan di `.env` sebagai `DATABASE_URL` dan masukkan `.env` ke `.gitignore`.
`DELETE FROM krs;`	Tanpa WHERE, semua KRS terhapus. Tulis WHERE lebih dulu, uji dengan SELECT yang sama.

Pola kesalahannya

Ketiganya tidak memunculkan error; kerugiannya baru terasa saat data bocor atau hilang.

Giliran kalian

ITERA

1 **Setengah jadi: relasi dosen**

Lengkapi: kolom `dosen_id INT _____ dosen(id)` di tabel `matkul`, dan query `SELECT k.nama, d.nama FROM matkul k _____ dosen d ON d.id = k._____;`

2 **Mandiri: endpoint KRS**

Tulis `GET /mahasiswa/{nim}/krs` yang mengembalikan mata kuliah yang diambil mahasiswa itu, 404 bila NIM tidak ada.

JAWABAN

Setengah jadi: REFERENCES, JOIN, dosen_id. Mandiri: cari mahasiswa dengan select where nim, 404 bila None, lalu select Matkul join krs where krs.mahasiswa_id == mhs.id.

Hands-on bertingkat

ITERA

Tingkat	Waktu	Yang dilatih	Sub-CPMK
Tingkat 1, Dasar	10′	SELECT, WHERE, ORDER BY, INSERT, UPDATE di psql	12.1
Tingkat 2, Menengah	15′	JOIN tiga tabel, GROUP BY, HAVING	12.1
Tingkat 3, Tantangan	10′	FastAPI dengan SQLAlchemy menggantikan list Python	12.2
Pengayaan	sisa	migrasi dengan Alembic	12.2

Aturan

Berkas awal ada di `02 Hands-on/P12 - PostgreSQL - Hands-on/starter`. Kerjakan berurutan dari tingkat 1, simpan bukti tangkapan layar, dan sebutkan alasan setiap langkah.

Presentasi di papan

ITERA

Tiga mahasiswa diundi untuk menunjukkan hasil hands-on tingkat 2 atau 3 di layar, masing-masing sekitar 5 menit, lalu menjawab satu pertanyaan.

Kriteria	Sangat baik 75--100	Baik 65--74	Cukup 55--64	Kurang <55
Ketepatan hasil	berjalan dan sesuai soal	satu salah kecil	ada bagian salah	sebagian besar keliru
Alasan tiap langkah	setiap baris dijelaskan fungsinya	sebagian besar dijelaskan	penjelasan kabur	tanpa penjelasan
Menjawab pertanyaan	tanpa melihat catatan	dengan sedikit bantuan	menjawab sebagian	tidak bisa menjawab

Yang tidak maju tetap bisa menambah nilai lewat pertanyaan atau koreksi yang tepat.

Exit ticket Pertemuan 12

ITERA

```
UPDATE mahasiswa SET nama = 'Rani Putri';
db.add(Matkul(kode="IF25-21030", nama="AI", sks=3))
```

1 Sub-CPMK 12.1

Baris SQL di atas berbahaya. Jelaskan akibatnya dan tulis versi yang benar untuk NIM 1241401*AB*.

2 Sub-CPMK 12.2

Baris Python di atas dijalankan di endpoint, tetapi data tidak muncul di PostgreSQL. Apa yang terlupa? Tulis query SQL yang menghitung jumlah mahasiswa per mata kuliah.

3 Refleksi

Tulis satu kalimat: bagian mana yang paling membingungkan hari ini, dan mengapa?

Soal 1 dan 2 dinilai 0, 1, atau 2: benar dengan alasan, benar tanpa alasan atau salah kecil, belum benar.

Yang dibawa pulang

ITERA

	Bagian	Inti
1	Basis data relasional dan PostgreSQL	Tabel berbaris dan berkolom dengan primary key; PostgreSQL di port 5432.
2	SQL dasar	INSERT menambah, SELECT dengan WHERE dan ORDER BY membaca, UPDATE dan DELETE selalu dengan WHERE.
3	Relasi dan JOIN	Foreign key menghubungkan tabel; banyak-ke-banyak lewat tabel penghubung; JOIN.
4	SQLAlchemy ORM	ORM: tabel jadi class, baris jadi object; session menjalankan query dan transaksi.
5	FastAPI dan PostgreSQL	Dependency memberi session per request; Pydantic membaca object ORM menjadi JSON.
6	Migrasi dan keamanan	Alembic mencatat perubahan skema; kredensial di .env, query selalu berparameter.

Sebelum Pertemuan 13: kerjakan Cek pemahaman Modul 12, finalkan rancangan tabel proyek kelompok, dan siapkan akun penyedia AI atau pasang Ollama untuk Pertemuan 13.

Ada pertanyaan?

Sampai jumpa di Pertemuan 13: AI pada Aplikasi Web

ITERA